Modern Qt Quick Applications

The concepts behind building, testing and shipping a Qt Quick app

Dennis Oberst

Contents

Welcome . . . 7
The app . . . 7
Quick start . . . 8
How to read this guide . . . 8
What you should know already . . . 9

I Foundations

1 The shape of a Qt Quick application . . . 13
1.1 QML composes, C++ implements . . . 13
1.2 Layers, with Qt's names . . . 13
1.3 Commands go down, notifications come up . . . 14
1.4 Dependencies point one way, and the build enforces it . . . 14
1.5 One place wires everything . . . 14
1.6 Interfaces at the seams . . . 15

2 Project layout . . . 17
2.1 A conventional top level . . . 17
2.2 Public headers in `include/<name>/` . . . 17
2.3 Separable code becomes its own library . . . 18
2.4 Group by feature, then by role . . . 18
2.5 Tests mirror the sources . . . 18
2.6 Include by module . . . 18

II Building

3 Build essentials . . . 21
3.1 Targets and usage requirements . . . 21
3.2 Qt's CMake API . . . 21
3.3 Static and dynamic libraries . . . 22
3.4 Presets: the build is one command . . . 22

4 Dependencies . . . 25
4.1 Three ways to get dependencies . . . 25
4.2 `find_package` is the stable interface . . . 25
4.3 vcpkg in manifest mode . . . 26

4.4 Where Qt comes from . . . 26
4.5 Reproducibility is control . . . 26

III Qt and QML

5 How Qt runs your code . . . 29
5.1 The event loop . . . 29
5.2 Properties and signals: the reactive contract . . . 29
5.3 Signals and slots . . . 30
5.4 Who owns an object . . . 30
5.5 Threads: hand work off, send results back . . . 30

6 QML done right . . . 31
6.1 Bindings, not assignments . . . 31
6.2 Components with a clear interface . . . 31
6.3 Typed QML that tools can check . . . 32
6.4 Lists come from models . . . 32
6.5 Positioning: anchors, layouts, positioners . . . 32
6.6 Keep JavaScript small . . . 33

7 Connecting C++ and QML . . . 35
7.1 QML modules and declarative registration . . . 35
7.2 Passing objects in: required properties . . . 35
7.3 Singletons for app-wide state . . . 36
7.4 Item models in C++ . . . 36
7.5 Lifetimes across the boundary . . . 37

8 Presentation patterns . . . 39
8.1 From MVC to MVVM . . . 39
8.2 MVVM . . . 39
8.3 Flux . . . 40
8.4 Choosing a pattern . . . 41

IV The application

9 Look and feel . . . 45
9.1 Your own Qt Quick Controls style . . . 45
9.2 Design tokens and dark mode . . . 46
9.3 Custom items and the scene graph . . . 46

10 What every real application needs . . . 49
10.1 Translations . . . 49
10.2 Standard paths and settings . . . 49

10.3 Saving without losing data . . . 50
10.4 Logging . . . 50
10.5 Crash diagnostics . . . 51

V Quality

11 Quality gates . . . 55
11.1 Warnings are errors, for your code . . . 55
11.2 Sanitizers . . . 55
11.3 Hardening . . . 56
11.4 Linters and formatters, one check . . . 56

12 Testing . . . 57
12.1 Test each layer without the UI . . . 57
12.2 Test QML components . . . 57
12.3 Tests must be deterministic . . . 58
12.4 Check your models . . . 58

VI Shipping

13 Shipping . . . 61
13.1 Install, then deploy . . . 61
13.2 One package per platform . . . 61
13.3 Licenses are part of the product . . . 62
13.4 CI builds what you ship . . . 62

14 Signing and distribution . . . 63
14.1 Why sign . . . 63
14.2 macOS: Developer ID and notarization . . . 63
14.3 Windows: Authenticode . . . 63
14.4 Signing in the pipeline . . . 63
14.5 App stores . . . 64

15 Mobile, briefly . . . 65
15.1 Same code, different device . . . 65
15.2 Building for a device . . . 65
15.3 Permissions . . . 65

VII Beyond

16 Learning Qt, and working with AI . . . 69
16.1 Concepts you own, details you look up . . . 69
16.2 Read in the right order . . . 69

16.3 Recognize stale Qt . . . 69
16.4 Give the AI current Qt knowledge . . . 69
16.5 Let the tools judge the result . . . 70
16.6 Own what you ship . . . 70

Appendices . . . 71

A Working with the template . . . 73
A.1 Rename the template . . . 73
A.2 Remove the demo . . . 73
A.3 Add a separable library . . . 73
A.4 Add a feature . . . 73
A.5 Add a dependency from vcpkg . . . 74
A.6 Ship a license notice for an asset . . . 74
A.7 Change the app icon . . . 74
A.8 Add a language . . . 74
A.9 Debug with more logging . . . 74

9.1 Light . . . 46
9.2 Dark . . . 46

Welcome

This guide teaches the concepts behind a modern Qt Quick application: how to structure it, build it, connect C++ to QML, test it and ship it. It does not repeat the Qt reference documentation. Each section explains one idea, shows it in a few lines of code, and points to the place where you can read more.

Everything here is real. The guide comes with a template, a small but complete application that applies every concept. When a section says **Template**, open that file and see the idea at work.

The app

The template's demo app is *Sketchbook*. You keep a list of sketches and draw on a canvas; circles, rectangles and triangles you draw snap into clean shapes.

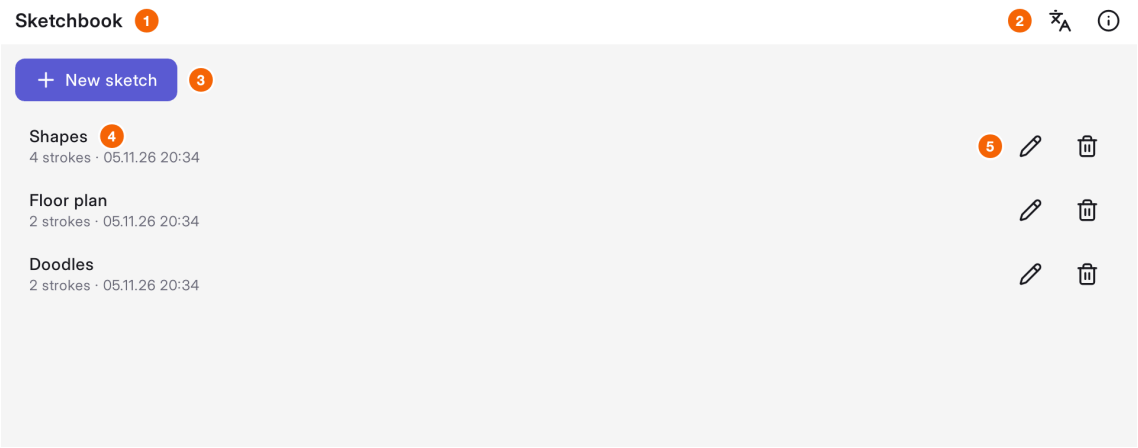


Figure 0.1: The sketchbook: a list of your sketches.

1. **Window header.** Shows the app name here, and the sketch's name on the canvas.
2. **Language and About.** Switch between English and German while the app runs (Chapter 10.). *About* shows the version, the exact build, and where the licenses are.
3. **New sketch.** A button from the app's own control style (Chapter 9.).
4. **A sketch.** Name, stroke count and date, delivered by a C++ list model (Chapter 7.).
5. **Rename and delete.** Commands that the view sends to its view model (Chapter 8.).

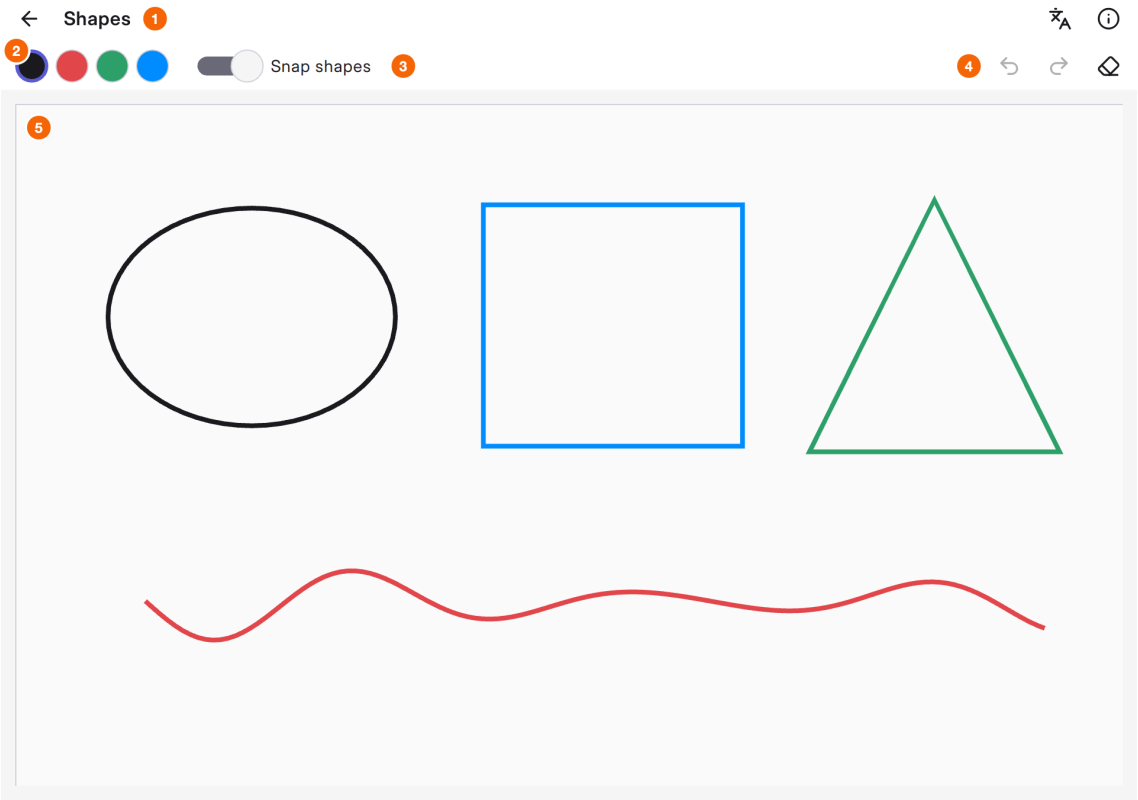


Figure 0.2: The canvas: draw, and shapes snap into place.

1. **Back and title.** Returns to the sketchbook.
2. **Inks.** Four pen colors.
3. **Snap shapes.** A plain C++ library recognizes what you drew (Chapter 2.).
4. **Undo, redo, clear.** Every edit can be undone, thanks to the Flux pattern (Chapter 8.).
5. **Paper.** A custom item that draws strokes with the scene graph (Chapter 9.). Scroll to move it, pinch or Ctrl+wheel to zoom.

Quick start

You need Git, CMake 3.31 or newer, Ninja, a C++23 compiler and Qt 6.12 or newer.

```
git clone --recurse-submodules https://github.com/<you>/<your-app>.git
cd <your-app>
export QT_ROOT_DIR=~/Qt/6.12.0/macos
cmake --workflow --preset dev
```

The last command configures, builds and runs all tests. The first run takes a few minutes because it builds the dependencies.

How to read this guide

Read it front to back once. The chapters build on each other: first the shape of an application, then the build, then Qt and QML, then quality and shipping. Afterwards, come back to single sections when you need them.

Each section ends with a short line:
Template: the file that shows the idea · **Read more:** one link that goes deeper
The guide explains *why* and *what*. The linked documentation explains *every option*.

What you should know already

You can write C++ and have seen a little QML. You do not need to know CMake or how Qt works inside; that is what this guide is for.

I Foundations

1 The shape of a Qt Quick application . 13
1.1 QML composes, C++ implements 13
1.2 Layers, with Qt's names 13
1.3 Commands go down, notifications come up . . . 14
1.4 Dependencies point one way, and the build enforces it 14
1.5 One place wires everything 14
1.6 Interfaces at the seams 15

2 Project layout 17
2.1 A conventional top level 17
2.2 Public headers in `include/<name>/` 17
2.3 Separable code becomes its own library 18
2.4 Group by feature, then by role 18
2.5 Tests mirror the sources 18
2.6 Include by module 18

1. The shape of a Qt Quick application

Before any detail, the big picture: which code goes where, and how the parts talk to each other. Every later chapter fills in one part of this picture.

1.1 QML composes, C++ implements

Qt Quick splits an application by language. C++ holds the state and the logic and offers them as properties, signals and methods. QML arranges the visual components and wires their properties to that state.

```
class CanvasStore : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    Q_PROPERTY(bool canUndo READ canUndo NOTIFY changed FINAL)

public:
    bool canUndo() const;
    Q_INVOKABLE void undo();

Q_SIGNALS:
    void changed();
};
```

```
ToolButton {
    text: qsTr("Undo")
    enabled: store.canUndo   // a binding: follows the store's state by itself
    onClicked: store.undo()  // a handler: sends the user's intent to C++
}
```

Moving that button, restyling it or putting it in a menu is a QML change that touches no logic. In Qt Widgets, layout and logic both live in C++, so the same change means editing and recompiling code. The split by language is also a split by responsibility, and that is the main reason to choose Qt Quick.
Template: `src/canvas/CanvasStore.hpp`, `src/canvas/CanvasPage.qml` · **Read more:** Qt Quick best practices

1.2 Layers, with Qt's names

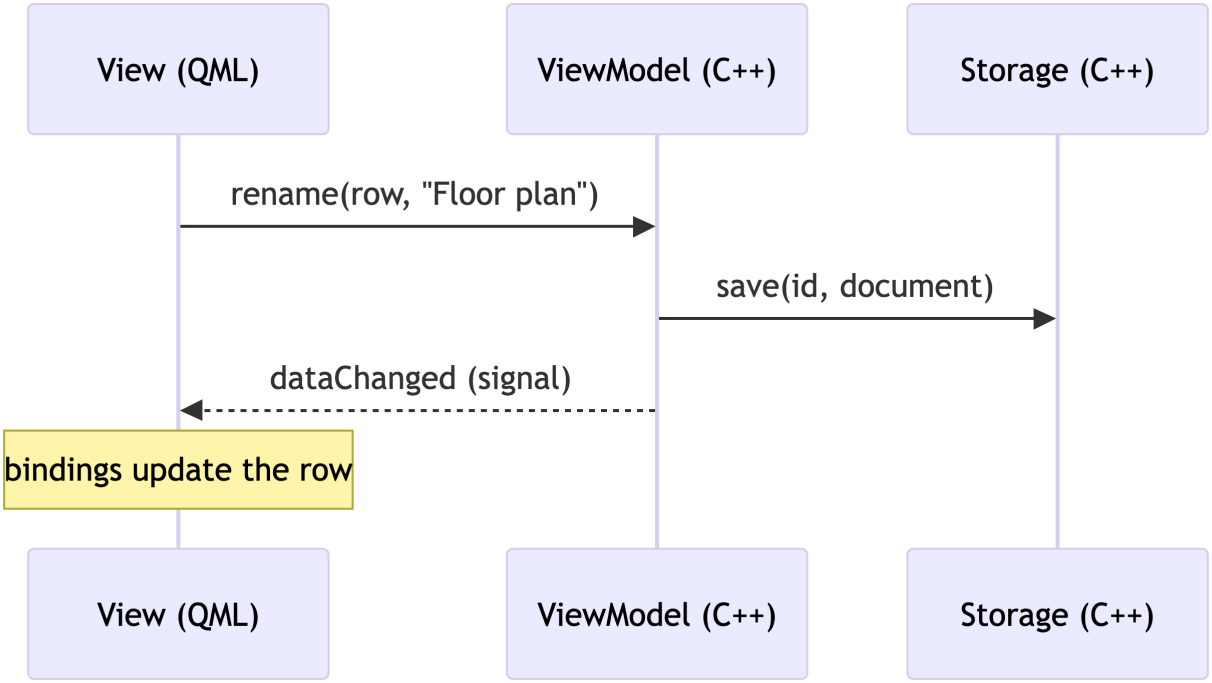
Most Qt Quick applications settle into four layers. Qt projects name them differently from textbooks, so here are both:

Layer	Name in Qt projects	Holds	Template example
Presentation	**View**	QML: layout, input, animation	`SketchbookPage.qml`
Application	**ViewModel** or **Controller**	QObjects that turn user intent into operations	`SketchbookViewModel`
Domain	**Core** or **Engine**	Plain C++ rules and algorithms	`shapes::Recognizer`
Infrastructure	**Backend**, **Storage**, **Network**	Files, sockets, devices	`SketchStorage`

Only the view is QML. Only the view model knows that QML exists. The core knows neither, which is why it is the easiest code to test and to reuse.

1.3 Commands go down, notifications come up

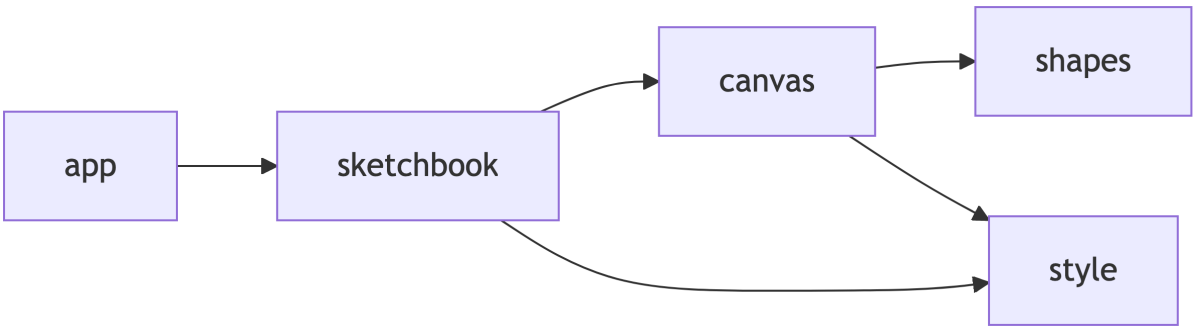
A layer calls the layer below it directly. The lower layer never calls back up; it emits a signal, and whoever cares listens. In QML, listening happens through bindings.



Because lower layers never know who listens, you can test them without a UI and replace the UI without touching them.

1.4 Dependencies point one way, and the build enforces it

Each layer and feature may only use what sits below it. Write that rule into the build: a library can only include headers of the libraries it links.




```
target_link_libraries(app_canvas PUBLIC shapes::shapes)
target_link_libraries(app_sketchbook PUBLIC app_canvas)
```

When `canvas` tries to include a `sketchbook` header, the compiler refuses. A broken rule fails the build on the spot, long before a code review would catch it.
Template: `src/CMakeLists.txt` and the `CMakeLists.txt` of each feature

1.5 One place wires everything

Some code has to know every concrete type and connect them. Keep that code in one place, `main()`, and nowhere else. It creates the long-lived objects, hands each its dependencies, and passes them to QML.

```
int main(int argc, char *argv[])
{
    QGuiApplication application(argc, argv);

    SketchStorage storage(dataDir);
    CanvasStore canvas;
    SketchbookViewModel sketchbook(storage, canvas);

    QQmlApplicationEngine engine;
    engine.setInitialProperties({
        { "sketchbook", QVariant::fromValue(&sketchbook) },
        { "canvas", QVariant::fromValue(&canvas) },
    });
    engine.loadFromModule("App", "Main");
    return application.exec();
}
```

Line 5
Objects receive their dependencies through the constructor. No globals.
Line 9
Created last, destroyed first: QML never sees an object that is already gone.
Every lifetime is visible in one function, and a test can build the same objects with fakes.
Template: `src/app/main.cpp`

1.6 Interfaces at the seams

Where your code meets the outside world (a network, a device, the clock), put an abstract class between them. The rest of the app depends on the interface, and a test passes in a fake.

```
class Transport
{
public:
    virtual ~Transport() = default;
    virtual void send(const QByteArray &payload) = 0;
};

class TcpTransport final : public Transport { /* the real network */ };
class FakeTransport final : public Transport { /* records what was sent */ };
```

Give the interface the plain noun, `Transport`, and let implementations say what they are: `TcpTransport`, `FakeTransport`. Avoid an `I` prefix such as `ITransport`; callers should not care that a type is abstract.

Add an interface when a second implementation exists, even if it is only a test fake. The template's storage has none: its tests write to a temporary folder instead.
Read more: C++ Core Guidelines, NL.5

2. Project layout

A good layout tells a newcomer where things are before they read a line of code. C++ has no official standard, but it has a widely used convention, and Qt Quick adds a few rules of its own.

2.1 A conventional top level

Base your layout on **Pitchfork**, the closest thing C++ has to a standard project layout. An application adapts it like this:

```
my-app/
├── CMakeLists.txt
├── CMakePresets.json
├── cmake/        build helpers
├── docs/         documentation
├── external/     third-party code you embed, unchanged
├── libs/         libraries that could live without the app
├── src/          the application
└── tests/        tests for src/
```

Pitchfork itself puts code either in `src/` or, split up, in `libs/`; an application with separable libraries commonly uses both, and adds `cmake/` for build helpers. Newcomers find code where they expect it, and the layout scales from one library to many.
Template: the repository root (`3rdparty/` plays the role of `external/`) · **Read more:** Pitchfork

2.2 Public headers in `include/<name>/`

Every library splits its headers in two. Public headers, the library's API, go into `include/<name>/`. Private headers stay next to the sources in `src/`.

```
core/
├── CMakeLists.txt
├── include/
│   └── core/
│       └── message.h        public: consumers include it
└── src/
    ├── message.cpp
    ├── peerregistry.h       private: invisible to consumers
    └── peerregistry.cpp
```

```
add_library(core STATIC src/message.cpp src/peerregistry.cpp)
target_include_directories(core
    PUBLIC include
    PRIVATE src
)
```

Line 3
Consumers get `include/` on their include path and write `#include <core/message.h>`.
Line 4
Only the library itself can include `peerregistry.h`.
The extra `core/` folder pays off at every include. `#include <core/message.h>` says where the header comes from and cannot collide with another library's `message.h`. Do this for every library, including those used only inside your project: the split shows at a glance what a library promises and what it keeps to itself.
Template: `libs/shapes/`, which declares its public headers as a `FILE_SET HEADERS`, the modern spelling of the same idea

2.3 Separable code becomes its own library

Ask of every piece of code: *would someone use this without our app?* An algorithm, a file parser or a protocol passes that test. It then becomes a library under `libs/`, with its own tests and, ideally, no Qt at all.

Such a library is the easiest code in the project to test, to reason about, and to reuse. Code that only serves one feature fails the test and stays inside that feature.

Template: `libs/shapes/`, a shape recognizer in plain C++. It does not link Qt, so a Qt include there fails to compile.

2.4 Group by feature, then by role

Organize the application by feature, not by file type. A feature folder holds everything the feature needs: its C++ types and its QML files, side by side, built as one QML module.

```
src/
├── app/            main.cpp, the main window
├── style/          the look: controls, colors, icons
├── canvas/         a small feature: a few files
└── contacts/       a grown feature, split by role
    ├── components/    reusable QML pieces: a search field, a delegate
    ├── pages/         screens that combine components
    ├── models/        item models
    └── controllers/   view models and controllers
```

Adding a feature adds a folder; removing it deletes one. Split by role only once a feature outgrows a single folder.

Template: `src/` · **Read more:** Qt 6 QML project architecture

2.5 Tests mirror the sources

`tests/canvas/` tests `src/canvas/`. When you look for a class's tests, you know where they are. A library under `libs/` keeps its tests in its own `tests/` folder, so it can move to another repository in one piece.

Template: `tests/` and `libs/shapes/tests/`

2.6 Include by module

Write Qt includes with their module, and your own libraries' includes with their folder:

```
#include <QtCore/QObject>
#include <QtQuick/QQuickItem>
#include <core/message.h>
```

Every include then tells the reader which module or library a type comes from, in the same form for Qt and for your own code.

II Building

3 Build essentials 21
3.1 Targets and usage requirements 21
3.2 Qt's CMake API 21
3.3 Static and dynamic libraries 22
3.4 Presets: the build is one command 22

4 Dependencies 25
4.1 Three ways to get dependencies 25
4.2 `find_package` is the stable interface 25
4.3 vcpkg in manifest mode 26
4.4 Where Qt comes from 26
4.5 Reproducibility is control 26

3. Build essentials

This chapter is not a CMake course. It covers the four ideas you need to read and extend a modern Qt build; the CMake and Qt documentation covers the rest.

3.1 Targets and usage requirements

A **target** is something CMake builds: a library or an executable. It carries its sources, its include paths and its dependencies. When one target links another, it inherits the other's **usage requirements**, and the keyword decides how far they travel.

```
add_library(shapes STATIC src/Recognizer.cpp)
target_include_directories(shapes PUBLIC include)

add_library(storage STATIC Storage.cpp)
target_link_libraries(storage
    PUBLIC shapes
    PRIVATE glaze::glaze
)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE storage)
```

Line 6
`storage`'s own headers use `shapes` types, so its users need `shapes` too.
Line 7
JSON handling is an implementation detail; `storage`'s users never see glaze.
Line 11
`app` gets `storage` and, through it, `shapes`, but not glaze's headers.
The rule of thumb: **PUBLIC** if the dependency appears in your headers, **PRIVATE** if it appears only in your `.cpp` files. Set everything on targets, never through global variables, and the build describes your architecture.
Read more: cmake-buildsystem(7)

3.2 Qt's CMake API

Qt adds a handful of commands on top of CMake. Three of them cover a Qt Quick application:

```
cmake_minimum_required(VERSION 3.31)
project(Sketchbook VERSION 0.1.0 LANGUAGES CXX)

find_package(Qt6 6.12 REQUIRED COMPONENTS Quick)
qt_standard_project_setup(REQUIRES 6.12)

qt_add_executable(app main.cpp)
qt_add_qml_module(app
    URI App
    QML_FILES Main.qml
    SOURCES AppController.cpp AppController.hpp
)
target_link_libraries(app PRIVATE Qt::Quick)
```

Line 5
Turns on Qt's code generators (such as moc) and the defaults of the Qt version you name.
Line 8
A **QML module**: a name to import (`URI`), its QML files and C++ sources, built and checked together.

You will meet `qt_add_qml_module()` again in Chapter 7.. It is the one command that ties C++ and QML together.
Template: `CMakeLists.txt`, `src/app/CMakeLists.txt` · **Read more:** Getting started with CMake (Qt)

3.3 Static and dynamic libraries

A library reaches your program in one of two ways.

A **static** library (`.a`, `.lib`) is copied into your executable when it is linked. The result is one file with everything inside.

A **dynamic** library (`.so`, `.dylib`, `.dll`) stays a separate file. The operating system loads it when the program starts, so it must be on the user's machine.

	Static	Dynamic
Joins the program	at link time	at start-up
On the user's machine	nothing extra	the library, from the system or shipped with the app
Memory	one copy per program	one copy shared by every program that uses it
Updates	rebuild the program	replace the library

The last two rows explain why systems share libraries such as the C runtime or OpenSSL: memory is used once, and a security fix reaches every program at once. For your app, the choice decides what you must ship (Chapter 13.) and, for LGPL libraries such as Qt, what the license requires (Section 13.3).

> ⚠ Works on my machine
>
> A dynamic library that exists on your computer but not on the user's makes the program fail at start-up. Every dynamic dependency is something you must ship or require.

3.4 Presets: the build is one command

`CMakePresets.json` records how to configure, build, test and package the project. Everyone, your CI included, runs the same command; nobody has to remember flags.

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/${presetName}",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug" }
    }
  ],
  "buildPresets": [{ "name": "dev", "configurePreset": "dev" }],
  "testPresets": [{ "name": "dev", "configurePreset": "dev" }],
  "workflowPresets": [
    {
      "name": "dev",
      "steps": [
        { "type": "configure", "name": "dev" },
        { "type": "build", "name": "dev" },
        { "type": "test", "name": "dev" }
      ]
    }
  ]
}
```

```
cmake --workflow --preset dev
```

Settings that belong to one machine, such as the path to your Qt or a build folder outside a synced directory, go into `CMakeUserPresets.json`. It can inherit from the shared presets and stays out of version control.
Template: `CMakePresets.json`, with `dev`, `asan`, `release` and `vcpkg-qt` workflows · **Read more:** cmake-presets(7)

4. Dependencies

Few applications are built from their own code alone. This chapter is about getting other people's code into your build, the same way on every platform.

4.1 Three ways to get dependencies

Every approach belongs to one of three families.

Family	Tools	Choose it when
Package managers	vcpkg, Conan	you have several dependencies, several platforms, and want pre-built binaries cached
CMake-native	`FetchContent`, CPM.cmake, `find_package` with system packages	you have a few small dependencies, or target one platform whose packages you trust
Source-level	copying the code in ("vendoring"), git submodules or subtrees	you patch the dependency, or develop it alongside your app

Package manager:

```
{
  "dependencies": ["catch2", "glaze"]
}
```

CMake-native:

```
include(FetchContent)
FetchContent_Declare(fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG 11.2.0
)
FetchContent_MakeAvailable(fmt)
```

Source-level:

```
git submodule add https://github.com/fmtlib/fmt.git external/fmt
```

```
add_subdirectory(external/fmt)
```

Source-level integration is common and often the right call. When you carry your own patches, or change a library as often as your app, having its code in your tree is the most flexible setup. The price: you update it by hand, and it builds with your project every time.

Package managers sit at the other end. They resolve versions, build each library once and cache the result, and work the same on every platform. The more dependencies and platforms you have, the more that pays.

4.2 `find_package` is the stable interface

However a dependency arrives, consume it the same way: `find_package()` plus the imported target it defines.

```
find_package(glaze CONFIG REQUIRED)
target_link_libraries(storage PRIVATE glaze::glaze)
```

vcpkg, Conan, a system package or a library you built yourself can all provide `glaze`. Switch the provider and none of your targets change.

4.3 vcpkg in manifest mode

With vcpkg, a `vcpkg.json` file in your repository lists the dependencies. The vcpkg toolchain file installs them while CMake configures:

```
"toolchainFile": "${sourceDir}/external/vcpkg/scripts/buildsystems/vcpkg.cmake"
```

What you get:

- **Pinned versions.** A vcpkg version (a `builtin-baseline` commit, or vcpkg itself as a git submodule) fixes the version of every library.
- **Debug and release builds** of each library by default; a triplet can limit it to release, which halves build time.
- **Control per library.** *Triplets* decide how libraries are built (static or dynamic, which platform, which flags); *overlay ports* let you patch one.
- **A binary cache.** Each library is built once and reused by every later build and every project.
- **Every platform with one setup**, from Windows, macOS and Linux to Android and iOS.

Template: `vcpkg.json`, `cmake/vcpkg-triplets/`, and vcpkg as a submodule in `3rdparty/vcpkg` · **Read more:** vcpkg manifest mode, triplets

4.4 Where Qt comes from

Qt is a dependency like any other, just a large one. You have two good options:

- **The Qt Online Installer.** Ready in minutes. Point CMake at it with `CMAKE_PREFIX_PATH`.
- **Build Qt yourself**, from source or through your package manager. A full build takes hours, so build only the modules you use. In return, every flag and every dependency of Qt is under your control, on every platform.

Template: the `dev` preset uses an installed Qt; the `vcpkg-qt` preset lets vcpkg build it

4.5 Reproducibility is control

A cross-platform app runs on machines you have never seen. Each platform brings its own surprises: Linux needs fontconfig, macOS frameworks, Windows the right runtime DLLs. Every library linked dynamically must exist on the user's machine at run time.

The answer is control. When one tool builds every dependency the same way on every platform, a build that works today still works tomorrow, on a colleague's machine, and in CI. Prebuilt binaries from elsewhere bring flags and dependencies you did not choose.

> Rule of thumb
>
> The more of your dependencies one tool builds for you, Qt included, the fewer surprises you meet when you ship.

III Qt and QML

5 How Qt runs your code 29
5.1 The event loop 29
5.2 Properties and signals: the reactive contract . . . 29
5.3 Signals and slots 30
5.4 Who owns an object 30
5.5 Threads: hand work off, send results back 30

6 QML done right 31
6.1 Bindings, not assignments 31
6.2 Components with a clear interface 31
6.3 Typed QML that tools can check 32
6.4 Lists come from models 32
6.5 Positioning: anchors, layouts, positioners 32
6.6 Keep JavaScript small 33

7 Connecting C++ and QML 35
7.1 QML modules and declarative registration 35
7.2 Passing objects in: required properties 35
7.3 Singletons for app-wide state 36
7.4 Item models in C++ 36
7.5 Lifetimes across the boundary 37

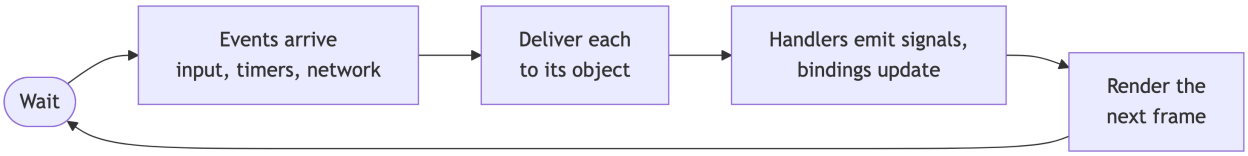
8 Presentation patterns 39
8.1 From MVC to MVVM 39
8.2 MVVM 39
8.3 Flux 40
8.4 Choosing a pattern 41

5. How Qt runs your code

QML bindings, signals and threads all rest on a few mechanisms in Qt's core. Understand these five and most of Qt's behavior stops being surprising.

5.1 The event loop

A Qt application is one loop. `QGuiApplication::exec()` starts it, and it runs until the app quits:



Every click, timer and update passes through this cycle, on the main thread, one at a time. (Qt Quick draws the frame itself on a render thread, Section 9.3.) That has one big consequence: **code that blocks the loop freezes the whole app.** No input, no animation, no repaint until your function returns.

```
QThread::sleep(2);                                  // frozen for 2 s
QTimer::singleShot(2000, this, &Uploader::retry);  // keeps running
```

Never wait inside the loop. Let a timer or a signal call you back, and move slow work to another thread (Section 5.5).
Read more: The event system

5.2 Properties and signals: the reactive contract

A `Q_PROPERTY` with a `NOTIFY` signal is state that others can observe. Change the value, emit the signal, and every QML binding that reads the property updates itself.

```
class CanvasStore : public QObject
{
    Q_OBJECT
    Q_PROPERTY(bool snapShapes READ snapShapes WRITE setSnapShapes
               NOTIFY snapShapesChanged FINAL)

public:
    bool snapShapes() const { return m_snapShapes; }
    void setSnapShapes(bool snap)
    {
        if (snap == m_snapShapes)
            return;
        m_snapShapes = snap;
        Q_EMIT snapShapesChanged();
    }

Q_SIGNALS:
    void snapShapesChanged();

private:
    bool m_snapShapes = true;
};
```

Line 11
No change, no signal. This avoids needless updates and binding loops.

Line 14
Every binding on `snapShapes` re-evaluates.

Mark a property `CONSTANT` when it never changes, and `FINAL` so that QML tooling can rely on its type.
Template: `src/canvas/CanvasStore.hpp` · **Read more:** The property system

5.3 Signals and slots

A signal announces that something happened, without knowing who listens. `connect()` wires it to a receiver:

```
connect(&m_canvas, &CanvasStore::edited,
        this, &SketchbookViewModel::saveOpenSketch);
connect(&m_canvas, &CanvasStore::edited,
        this, [this] { scheduleBackup(); });
```

Use member-function pointers, as above: the compiler checks that signal and slot fit. The third argument of the lambda version, the **context object**, matters: when it is destroyed, the connection ends, and the lambda runs in that object's thread.

> **i** `Q_SIGNALS` and `Q_EMIT`
>
> The template writes `Q_SIGNALS`, `Q_SLOTS` and `Q_EMIT` instead of `signals`, `slots` and `emit`. The short forms are macros, and they break third-party headers that use those words as names.

Read more: Signals and slots

5.4 Who owns an object

A QObject can have a **parent**. When the parent is destroyed, it destroys its children. Objects without a parent belong to whoever created them: a stack variable, a member, or a `std::unique_ptr`.

```
auto *timer = new QTimer(this);                      // the parent deletes it
auto translator = std::make_unique<QTranslator>();   // the unique_ptr deletes it
SketchStorage storage(dataDir);                      // the scope deletes it
```

Give every object exactly one owner. With two owners, such as a parent *and* a `unique_ptr`, the parent may delete the object first, and the `unique_ptr` later deletes it a second time and crashes.
Read more: Object trees and ownership

5.5 Threads: hand work off, send results back

Each QObject belongs to the thread that created it. The GUI belongs to the main thread and must only be touched there. Slow work, such as writing a file or parsing a large document, runs on another thread and reports back with a signal.

```
void Storage::save(QString path, Document document)
{
    QtConcurrent::run(&m_writer,
        [path, document = std::move(document)] { writeJson(path, document); })
        .then(this, [this] { Q_EMIT saved(); });
}
```

Line 3
The worker gets its own copy of the data, or data that nobody changes. Then it needs no locks.

Line 5
With a context object, the continuation runs in that object's thread: back on the GUI thread.

For long-lived background work, such as a network connection, move a worker object to a `QThread` and talk to it through signals. Qt Quick itself renders on a separate thread, too (Section 9.3).
Template: `src/sketchbook/SketchStorage.cpp` saves on a one-thread pool, so writes keep their order · **Read more:** Threads and QObjects, The eight rules of multithreaded Qt

6. QML done right

QML looks like a layout language, but it is a language for relationships: this item is half as wide as that one, this button is enabled while that list is not empty. Writing good QML means stating relationships and keeping everything else in C++.

6.1 Bindings, not assignments

A property written with a colon is a **binding**. It stays true: whenever the right side changes, QML re-evaluates it.

```
Rectangle {
    id: panel
    width: parent.width / 2          // a binding: follows the parent, forever

    MouseArea {
        anchors.fill: parent
        onClicked: panel.width = 300   // an assignment: replaces the binding
    }
}
```

After the click, `panel` has a fixed width of 300, and resizing the window no longer resizes it. Most layout bugs in QML come from an assignment where a binding was meant. If a value should follow something, bind it. If the user changes it, change the *source* of the binding, usually a property in C++, instead of overwriting the binding.
Read more: Property binding

6.2 Components with a clear interface

A reusable component works like a function: data goes in through properties, intent comes out through signals. It never reaches into its parent or into ids defined elsewhere.

```
AbstractButton {
    id: swatch

    required property color ink   // input: whoever uses Swatch must set it
    signal picked(color ink)       // output: reports what happened

    implicitWidth: 28
    implicitHeight: 28
    onClicked: swatch.picked(swatch.ink)

    background: Rectangle {
        radius: width / 2
        color: swatch.ink
    }
}
```

The component only reports that it was picked; its user decides what that means:

```
Swatch {
    ink: "#e5484d"
    onPicked: ink => page.penColor = ink
}
```

A component written this way works anywhere, and a test can create it on its own.

Template: the `Swatch` inline component in `src/canvas/CanvasPage.qml` · **Read more:** QML component design

6.3 Typed QML that tools can check

QML can be as loose as JavaScript or as strict as C++. Write it strict, and qmllint and the QML compiler can check every name and type at build time; the compiler can then turn your bindings into C++.

```
pragma ComponentBehavior: Bound                    // bound to this file

Page {
    id: root

    required property SketchbookViewModel viewModel   // typed and required

    function rename(row: int, name: string): void {   // typed signature
        root.viewModel.rename(row, name)               // qualified lookup
    }
}
```

Each line replaces a guess with a fact the tools can check: no name that "should exist" somewhere above, no untyped argument, no lookup that depends on where the component is used.

Make qmllint fail on any warning and run it in CI, so a typo fails like a C++ typo does.

Template: `.qmllint.ini` · **Read more:** qmllint

6.4 Lists come from models

A `ListView` creates delegates only for the rows that are visible, and drops them as they scroll out of view (`reuseItems: true` recycles them instead). Its data comes from a **model**, and each delegate receives the data of its row as required properties:

```
ListView {
    id: list
    model: root.viewModel.model

    delegate: ItemDelegate {
        required property int index
        required property string name   // one property per model role
        required property int strokeCount

        width: list.width
        text: name
        onClicked: root.viewModel.open(index)
    }
}
```

Expose collections as models (Section 7.4), never as a property holding a whole list: a model can report "row 3 changed", a list property can only say "everything changed".

Template: `src/sketchbook/SketchbookPage.qml` · **Read more:** Models and views in Qt Quick

6.5 Positioning: anchors, layouts, positioners

Qt Quick offers three ways to place items. Pick one per container.

Tool	Use it for
Anchors (`anchors.fill`, `anchors.centerIn`)	relating an item to its parent or a sibling
Layouts (`RowLayout`, `ColumnLayout`, `GridLayout`)	sharing space among items that resize
Positioners (`Row`, `Column`, `Grid`)	stacking items of fixed size

```
RowLayout {
    anchors.fill: parent          // anchors place the layout
    Label {
        text: qsTr("Sketchbook")
        Layout.fillWidth: true    // takes the free space
    }
    ToolButton { icon.source: "info.svg" }
}
```

Read more: Positioning in Qt Quick

6.6 Keep JavaScript small

JavaScript in QML is glue. A handler calls one method; an expression formats one value.

```
onClicked: root.viewModel.remove(index)   // good: intent goes to C++
```

Logic written in QML escapes the compiler's type checks, cannot be tested from C++, and mixes with layout code. When a handler grows beyond a few lines, the code belongs in a C++ view model.

7. Connecting C++ and QML

C++ holds the state, QML shows it (Section 1.1). This chapter is about the seam between them: how C++ types become QML types, how objects cross over, and who owns them afterwards.

7.1 QML modules and declarative registration

Mark a C++ class `QML_ELEMENT`, list it in a QML module, and QML can use it under the module's name. There is no registration code in `main()`.

```cpp
#include <QtCore/QObject>
#include <QtQmlIntegration/qqmlintegration.h>

class CanvasStore : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    // properties, methods, signals
};
```

```
qt_add_qml_module(app_canvas
    URI App.Canvas
    STATIC
    SOURCES CanvasStore.cpp CanvasStore.hpp
    QML_FILES CanvasPage.qml
)
```

```
import App.Canvas   // CanvasStore and CanvasPage are now types
```

The build generates the registration, and the QML tools learn every type, property and method of the module. That is what makes typed QML (Section 6.3) possible.

> ⚠ Static modules need their plugin
>
> A `STATIC` QML module registers itself only when its plugin target is linked. Link `app_canvasplugin`, not just `app_canvas`, into the executable and into QML tests.

Template: `src/canvas/CMakeLists.txt` · **Read more:** Defining QML types from C++

7.2 Passing objects in: required properties

The main QML file declares what it needs. `main()` provides it before loading the file:

```
ApplicationWindow {
    id: window
    required property SketchbookViewModel sketchbook
    required property CanvasStore canvas

    CanvasPage { store: window.canvas }   // pages get their objects the same way
}
```

```
engine.setInitialProperties({
    { "sketchbook", QVariant::fromValue(&sketchbook) },
```

```
    { "canvas", QVariant::fromValue(&canvas) },
});
engine.loadFromModule("App", "Main");
```

Every dependency is visible in the component's interface and has a type the tools can check. A test can load any page on its own and hand it a fake.

Mark such classes `QML_UNCREATABLE("Created in main()")`, so QML cannot create a second, unconnected instance by accident.

> ⚠ Never use context properties
>
> `rootContext()->setContextProperty("canvas", &canvas)` injects a name that no tool can see or type-check. It is the Qt 5 way. Required properties replace it.

Template: `src/app/main.cpp`, `src/app/Main.qml` · **Read more:** Exposing C++ state to QML

7.3 Singletons for app-wide state

Some state is needed by every view: the language, the build information. Passing it through every component would be noise. Make it a QML singleton, but create the instance yourself:

```
class AppController : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    QML_SINGLETON
    QML_UNCREATABLE("Created in main()")
    Q_PROPERTY(QString language READ language WRITE setLanguage
               NOTIFY languageChanged FINAL)
    // ...
};
```

```
AppController controller(logFile);
engine.setExternalSingletonInstance("App", "AppController", &controller);
```

```
onClicked: AppController.language = "de"
```

Unlike a classic singleton, this one is not a global in C++. `main()` constructs it with its real dependencies, and a test can construct another. Keep singletons rare: most objects belong to a feature and arrive as required properties.

Template: `src/app/AppController.hpp`

7.4 Item models in C++

Collections reach QML as item models. Subclass `QAbstractListModel`, name your roles, and report every change as precisely as you can:

```
int SketchListModel::rowCount(const QModelIndex &parent) const
{
    return parent.isValid() ? 0 : static_cast<int>(m_entries.size());
}

QVariant SketchListModel::data(const QModelIndex &index, int role) const
{
    if (!checkIndex(index, CheckIndexOption::IndexIsValid))
        return {};
    const auto &entry = m_entries.at(index.row());
    switch (role) {
    case NameRole: return entry.name;
```

```
    case StrokeCountRole: return entry.strokeCount;
    default: return {};
    }
}

QHash<int, QByteArray> SketchListModel::roleNames() const
{
    return { { NameRole, "name" }, { StrokeCountRole, "strokeCount" } };
}

void SketchListModel::prepend(SketchEntry entry)
{
    beginInsertRows({}, 0, 0);
    m_entries.insert(m_entries.begin(), std::move(entry));
    endInsertRows();
}
```

Line 20
The role names become the delegate's required properties (Section 6.4).

Line 25
"One row arrives at the top": the view adds one delegate and keeps its scroll position.

Prefer `beginInsertRows`, `beginRemoveRows` and `dataChanged` over a model reset. A reset tells the view to throw everything away, so it loses scroll position, selection and any running animation.

Template: `src/sketchbook/SketchListModel.cpp` · **Read more:** Model/view programming

7.5 Lifetimes across the boundary

QML holds plain pointers to your C++ objects. Make sure those objects outlive the engine: create them before it in `main()`, and C++ destroys them after it (Section 1.5).

One rule surprises almost everyone. When a `Q_INVOKABLE` method returns a `QObject *` that has no parent, **QML takes ownership** of it and may delete it during garbage collection.

```
Q_INVOKABLE Document *createDocument();   // a parentless result now belongs to QML
```

Give such objects a parent, or call `QQmlEngine::setObjectOwnership(object, QQmlEngine::CppOwnership)` before returning them.

Read more: Data ownership

8. Presentation patterns

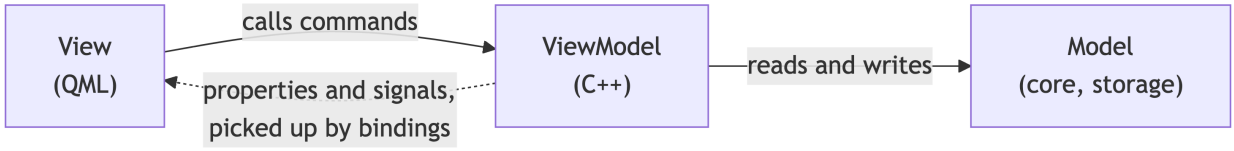
The layers of Section 1.2 say *where* code lives. A presentation pattern says how the view and the code behind it talk. Qt Quick works best with two of them: MVVM for most screens, Flux for documents with a history.

8.1 From MVC to MVVM

Three patterns share the same goal, keeping the view free of logic, and differ in who updates the view:

Pattern	How the view gets updated	Where you meet it in Qt
MVC	the view reads the model; a controller handles input	Qt's model/view classes: an item model, a view, a delegate
MVP	a presenter pushes every value into a passive view	many Qt Widgets applications
MVVM	the view *binds* to a view model; nothing pushes	Qt Quick

MVVM fits Qt Quick because bindings do the updating. The view model only has to change its properties and emit their signals.



8.2 MVVM

A view model offers **state** as properties and **commands** as invokable methods. The view binds to the state and calls the commands. It decides nothing.

```
class SketchbookViewModel : public QObject
{
    Q_OBJECT
    QML_ELEMENT
    QML_UNCREATABLE("Created in main()")
    Q_PROPERTY(SketchListModel *model READ model CONSTANT FINAL)
    Q_PROPERTY(QString openName READ openName NOTIFY openNameChanged FINAL)

public:
    Q_INVOKABLE void create();
    Q_INVOKABLE void rename(int row, const QString &name);
    Q_INVOKABLE void remove(int row);
    // ...
};
```

Line 6
State: the list of sketches and the name of the open one.

Line 10
Commands: what the user can do. Validation and storage happen behind them.

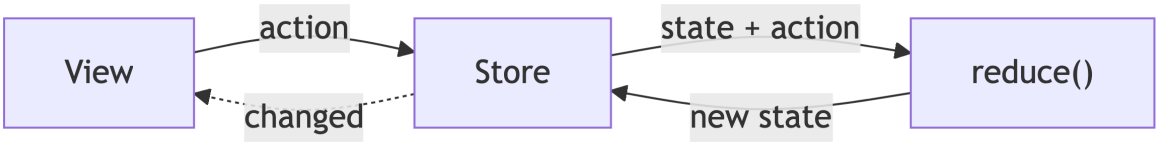
```
Button {
    text: qsTr("New sketch")
```

```
    onClicked: root.viewModel.create()
}
```

The view model contains no QML and no visual types, so a plain Qt Test can drive it: call `create()`, then check the model's row count (Section 12.1).
Template: `src/sketchbook/`

8.3 Flux

Flux sends every change along one path. The view dispatches an **action**; a pure function, the **reducer**, computes the next state from the current state and the action; the **store** keeps the result and notifies the views.



In C++, actions are a `std::variant` and the reducer visits it:

```
using StrokePtr = std::shared_ptr<const Stroke>;
using Sketch = std::vector<StrokePtr>;

struct AddStroke { StrokePtr stroke; };
struct Clear {};
using Action = std::variant<AddStroke, Clear>;

template <typename... Fs>
struct Overloaded : Fs... { using Fs::operator()...; };

Sketch reduce(Sketch sketch, const Action &action)
{
    std::visit(Overloaded{
        [&](const AddStroke &add) { sketch.push_back(add.stroke); },
        [&](Clear) { sketch.clear(); },
    }, action);
    return sketch;
}
```

Line 1
A stroke never changes once drawn, so every state shares it instead of copying it.
Line 11
Pure: the same sketch and action always give the same result. A new action type that `reduce()` forgets to handle fails to compile.

Because each state is a value, keeping old states gives undo and redo almost for free:

```
void CanvasStore::dispatch(const Action &action)
{
    auto next = reduce(m_history[m_current], action);
    m_history.resize(m_current + 1);
    m_history.push_back(std::move(next));
    m_current = m_history.size() - 1;
    Q_EMIT changed();
}

void CanvasStore::undo()
{
    if (m_current > 0) {
        --m_current;
        Q_EMIT changed();
```

```
    }
}
```

Line 4
A new edit drops the states that were undone.

Line 13
Undo moves a cursor back. No action needs an inverse.

Template: `src/canvas/Sketch.hpp`, `src/canvas/CanvasStore.cpp`

8.4 Choosing a pattern

Your screen	Pattern
Lists, forms, settings: state that maps onto controls	MVVM
Editors and documents whose history matters: undo, autosave, replay	Flux
Pure presentation: a dialog that only shows values	plain QML with required properties

One app can mix them, one pattern per feature. The template does exactly that: the sketchbook is MVVM, the canvas is Flux.

For very large documents, copying even a vector of pointers per edit becomes expensive. That is the point to look at persistent data structures, which share unchanged parts automatically.

IV The application

9 Look and feel 45
9.1 Your own Qt Quick Controls style 45
9.2 Design tokens and dark mode 46
9.3 Custom items and the scene graph 46

10 What every real application needs . 49
10.1 Translations 49
10.2 Standard paths and settings 49
10.3 Saving without losing data 50
10.4 Logging 50
10.5 Crash diagnostics 51

9. Look and feel

An application's look belongs in one place, not scattered over its pages. This chapter covers a custom control style, design tokens with light and dark themes, and what to do when no built-in item can draw what you need.

9.1 Your own Qt Quick Controls style

Every Qt Quick control comes in two halves. A **template** in `QtQuick.Templates` holds the behavior: what pressed, checked and focused mean. A **style** draws it. Writing your own style means drawing the controls you use on top of their templates:

```
import QtQuick
import QtQuick.Templates as T

T.Button {
    id: control

    implicitWidth: Math.max(implicitBackgroundWidth,
                            implicitContentWidth + leftPadding + rightPadding)
    implicitHeight: Math.max(implicitBackgroundHeight,
                             implicitContentHeight + topPadding + bottomPadding)
    padding: 8
    horizontalPadding: 16

    contentItem: Text {
        text: control.text
        font: control.font
        color: control.highlighted ? Theme.accentText : Theme.text
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }

    background: Rectangle {
        implicitHeight: 36
        radius: Theme.radius
        color: control.highlighted ? Theme.accent
             : control.down ? Theme.pressed : Theme.surface
    }
}
```

Put these files into their own QML module and let it fall back to the Basic style for every control you do not restyle:

```
qt_add_qml_module(app_style
    URI AppStyle
    IMPORTS QtQuick.Controls.Basic
    QML_FILES Button.qml Theme.qml   # and every other control you style
)
```

Views then write `import AppStyle` instead of `import QtQuick.Controls`. Because the style is chosen at compile time, qmllint and the QML compiler know the exact type of every control. Basic is the style made for customization; the native macOS and Windows styles are not.
Template: `src/style/` · **Read more:** Customizing Qt Quick Controls

9.2 Design tokens and dark mode

Colors, spacing, radii and fonts are **design tokens**. Keep them in one QML singleton and let every control read from it, never from literals:

```
pragma Singleton
import QtQuick

QtObject {
    readonly property bool dark: Application.styleHints.colorScheme ===
                                 Qt.ColorScheme.Dark

    readonly property color window: dark ? "#17171c" : "#f6f6f8"
    readonly property color surface: dark ? "#22222a" : "#ffffff"
    readonly property color pressed: dark ? "#393944" : "#dedee6"
    readonly property color text: dark ? "#ececf1" : "#1c1c22"
    readonly property color accent: "#5b5bd6"
    readonly property color accentText: "#ffffff"
    readonly property int radius: 8
}
```

The tokens follow the system's color scheme, so the whole app switches between light and dark with no code in the views. Mark the file as a singleton in CMake, before `qt_add_qml_module()`, with `set_source_files_properties(Theme.qml PROPERTIES QT_QML_SINGLETON_TYPE TRUE)`.

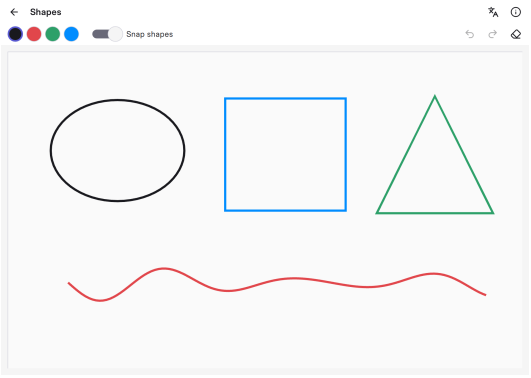


Figure 9.1: Light

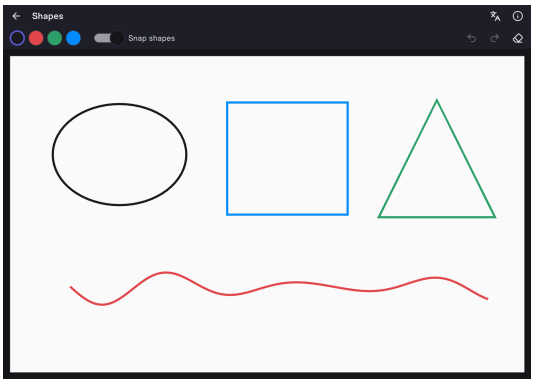


Figure 9.2: Dark

Set the window's `palette` from the tokens as well. The palette reaches every control, including the ones you did not restyle, so they match too.
Template: `src/style/Theme.qml`, `src/style/ApplicationWindow.qml`

9.3 Custom items and the scene graph

When no built-in item can draw what you need, write one in C++. Qt Quick offers several levels:

Option	How it draws	Use it for
Qt Quick Shapes	vector paths declared in QML	static icons and diagrams
`QQuickPaintedItem`	`QPainter` into a texture	simple custom drawing that rarely changes
Scene graph item	geometry nodes on the GPU	content that changes often, such as strokes or charts
`QQuickRhiItem`	your own GPU commands and shaders	3D and heavy rendering

A scene graph item sets `ItemHasContents` and builds its geometry in `updatePaintNode()`:

```
LineItem::LineItem(QQuickItem *parent)
    : QQuickItem(parent)
{
    setFlag(ItemHasContents);
}

QSGNode *LineItem::updatePaintNode(QSGNode *oldNode, UpdatePaintNodeData *)
{
    auto *node = static_cast<QSGGeometryNode *>(oldNode);
    if (!node) {
        node = new QSGGeometryNode;
        auto *geometry =
            new QSGGeometry(QSGGeometry::defaultAttributes_Point2D(), 0);
        geometry->setDrawingMode(QSGGeometry::DrawLineStrip);
        node->setGeometry(geometry);
        node->setFlag(QSGNode::OwnsGeometry);
        auto *material = new QSGFlatColorMaterial;
        material->setColor(Qt::black);
        node->setMaterial(material);
        node->setFlag(QSGNode::OwnsMaterial);
    }
    auto *geometry = node->geometry();
    geometry->allocate(static_cast<int>(m_points.size()));
    auto *vertices = geometry->vertexDataAsPoint2D();
    for (qsizetype i = 0; i < m_points.size(); ++i)
        vertices[i].set(float(m_points[i].x()), float(m_points[i].y()));
    node->markDirty(QSGNode::DirtyGeometry);
    return node;
}
```

Line 10
The first call creates the node; later calls receive it back and update it.

Line 22
Only the vertices change on later calls. The node and its material stay.

`updatePaintNode()` runs on the **render thread** while the GUI thread waits. It may read the item's members safely, but must only build scene graph nodes; it must not touch QML or other QObjects. To redraw, call `update()` from the GUI thread, and Qt schedules the next call.

Reuse nodes between frames and change only what changed. That is where a scene graph item beats a painted item, which repaints its whole texture every time.

Template: `src/canvas/StrokeItem.cpp`, which builds one node per stroke and keeps the nodes of strokes that did not change · **Read more:** The Qt Quick scene graph

10. What every real application needs

A demo can skip these. A real application cannot: it speaks the user's language, keeps its files in the right place, never loses data, and leaves a trail when something goes wrong.

10.1 Translations

Wrap every string the user sees: `qsTr()` in QML, `tr()` in C++. Let Qt handle plurals with `%n`, and fill in values with `arg()` instead of joining strings, so translators can reorder the sentence.

```
Label { text: qsTr("%n stroke(s)", "", strokeCount) }   // "1 stroke", "4 strokes"
Label { text: qsTr("Version %1").arg(Application.version) }
```

CMake extracts the strings into `.ts` files for translators and compiles them into the app:

```
qt_standard_project_setup(REQUIRES 6.12
    I18N_SOURCE_LANGUAGE en
    I18N_TRANSLATED_LANGUAGES de
)
qt_add_translations(app TS_FILE_BASE app TS_FILE_DIR i18n)
```

Run the `update_translations` target to refresh the `.ts` files, translate them with Qt Linguist, and rebuild.

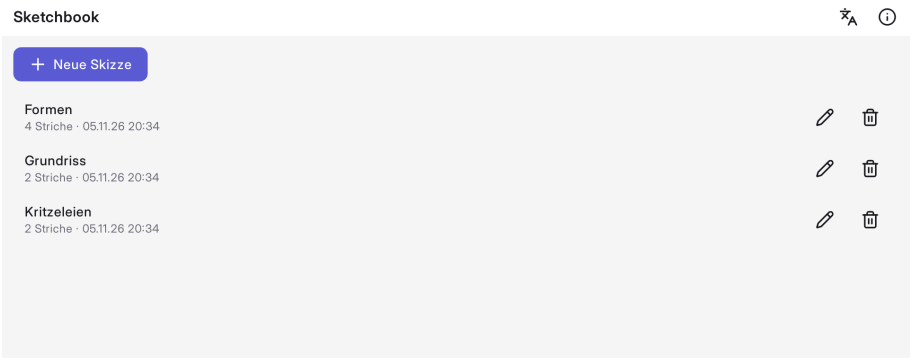


Figure 10.1: The same sketchbook in German.

Switching the language while the app runs takes two steps: install a new `QTranslator`, then call `QQmlEngine::retranslate()` so every `qsTr()` binding re-evaluates.
Template: `AppController::setLanguage()` in `src/app/AppController.cpp`, and `i18n/` · **Read more:** Internationalization with Qt

10.2 Standard paths and settings

Never hardcode a path. Each platform has its own place for application data, configuration and caches, and `QStandardPaths` knows them all:

```
const auto dataDir =
    QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation);
// macOS:   ~/Library/Application Support/<app>
// Windows: C:/Users/<user>/AppData/Local/<app>
// Linux:   ~/.local/share/<app>
```

The application and organization names you set on `QGuiApplication` become part of these paths, so set them first thing in `main()`.

Small preferences, such as the last language or the window size, go into `QSettings`. It stores them in the platform's native place:

```
QSettings settings;
settings.setValue("language", "de");
const auto language = settings.value("language", "en").toString();
```

Template: `src/app/main.cpp` (the template does not persist preferences yet) · **Read more:** QStandardPaths, QSettings

10.3 Saving without losing data

A crash or a full disk in the middle of a write leaves a half-written file, and the user's work is gone. Write to a temporary file instead, and replace the old file only when the new one is complete. `QSaveFile` does exactly that:

```
QSaveFile file(path);
if (!file.open(QIODevice::WriteOnly) || file.write(json) < 0 || !file.commit())
    qCWarning(lcStorage) << "Cannot save" << path << file.errorString();
```

Line 2

Until `commit()` succeeds, the old file stays untouched.

Save on a worker thread (Section 5.5) so large files never freeze the UI.

Template: `src/sketchbook/SketchStorage.cpp` · **Read more:** QSaveFile

10.4 Logging

Log through a **category** per module. You can then switch the debug output of one module on at run time, without rebuilding:

```
Q_STATIC_LOGGING_CATEGORY(lcStorage, "app.storage")

qCDebug(lcStorage) << "Saving" << path;
qCWarning(lcStorage) << "Cannot save" << path << file.errorString();
```

```
QT_LOGGING_RULES="app.storage.debug=true" ./Sketchbook
```

Write the log to a file as well, with a message handler (`qInstallMessageHandler`). Start every log with the version, the commit, the compiler and the operating system: a log file from a user is then enough to reproduce their setup.

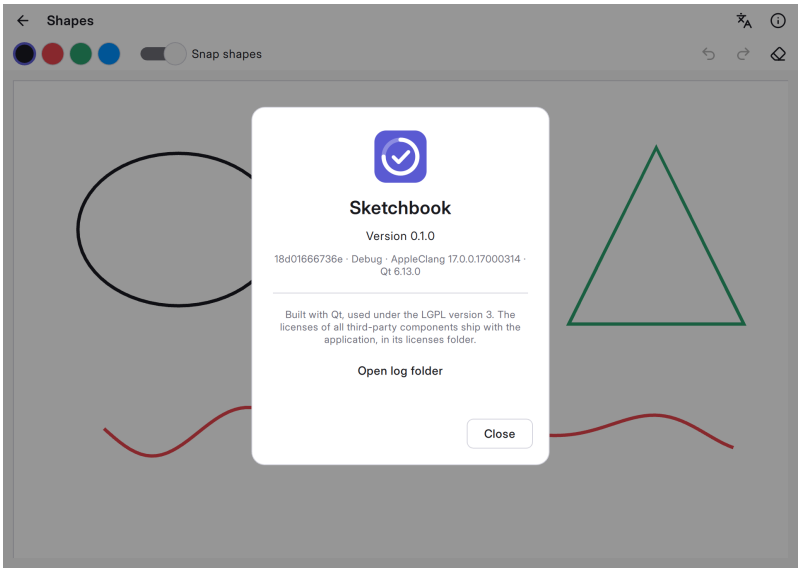


Figure 10.2: The About dialog shows the same build details and opens the log folder.

Template: `src/app/Logging.cpp` · **Read more:** QLoggingCategory

10.5 Crash diagnostics

When an exception escapes, C++ calls `std::terminate()`, and by default the app simply disappears. Install a handler that logs a stack trace first:

```
std::set_terminate([] {
    qCCritical(lcCrash).noquote()
        << "Uncaught exception\n"
        << QString::fromStdString(cpptrace::generate_trace().to_string());
    std::abort();
});
```

A stack trace is only readable if the binary still knows its function names. Build releases as `RelWithDebInfo` (optimized, with debug information), and keep the symbols when you deploy.

Crashes such as segmentation faults are different: code that runs after memory corruption can do further damage. Leave them to the operating system's crash reporter, or to a crash reporting service.

Template: `src/app/Logging.cpp` · **Read more:** cpptrace

V

Quality

11 Quality gates 55
11.1 Warnings are errors, for your code 55
11.2 Sanitizers 55
11.3 Hardening 56
11.4 Linters and formatters, one check 56

12 Testing 57
12.1 Test each layer without the UI 57
12.2 Test QML components 57
12.3 Tests must be deterministic 58
12.4 Check your models 58

11. Quality gates

Compilers and tools find whole classes of bugs before any user does, if you let them. Set these gates up on day one; adding them to a grown codebase means fixing hundreds of findings at once.

11.1 Warnings are errors, for your code

Turn on a strict warning set for your own targets:

```
if(MSVC)
    target_compile_options(core PRIVATE /W4 /permissive-)
else()
    target_compile_options(core PRIVATE
        -Wall -Wextra -Wpedantic -Wshadow -Wconversion)
endif()
```

Then make every warning an error, in the preset rather than in `CMakeLists.txt`, so you can switch it off locally when you need to:

```
"cacheVariables": { "CMAKE_COMPILE_WARNING_AS_ERROR": true }
```

Apply the warnings to your code only. Headers of imported targets, such as Qt and your vcpkg libraries, count as system headers, so their warnings stay out of your build.
Template: `cmake/AppTargetDefaults.cmake`, which applies warnings, hardening and sanitizers to every first-party target with one call

11.2 Sanitizers

Sanitizers make the compiler add checks to your program. When a check fails, the program stops and points at the bug, instead of corrupting memory silently and crashing hours later.

Sanitizer	Finds
AddressSanitizer (ASan)	use after free, buffer overflows, leaks (on Linux)
UndefinedBehaviorSanitizer (UBSan)	signed overflow, invalid casts, misaligned access
ThreadSanitizer (TSan)	data races between threads

Give them their own preset and run the whole test suite under it regularly:

```
{
  "name": "asan",
  "inherits": "dev",
  "cacheVariables": {
    "CMAKE_CXX_FLAGS": "-fsanitize=address,undefined -fno-omit-frame-pointer"
  }
}
```

A report names the bug and the line:

```
==4711==ERROR: AddressSanitizer: heap-use-after-free on address 0x602000001a30
READ of size 8 at 0x602000001a30 thread T0
    #0 0x104a1c3f8 in StrokeItem::updatePaintNode(...) StrokeItem.cpp:118
```

ThreadSanitizer needs a build of its own; it cannot be combined with ASan. MSVC supports AddressSanitizer only.
Template: `cmake --workflow --preset asan` · **Read more:** AddressSanitizer

11.3 Hardening

Sanitizers are too slow for releases. Hardening is their cheap cousin: checks that stay on in the shipped app and turn silent memory corruption into a clean crash.

- Standard library assertions: `_GLIBCXX_ASSERTIONS` (libstdc++), `_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_FAST` (libc++)
- Buffer checks: `_FORTIFY_SOURCE=3`
- Stack protection: `-fstack-protector-strong`
- On MSVC: `/sdl`, and `/guard:cf` for both the compiler and the linker

Template: `cmake/AppTargetDefaults.cmake` · **Read more:** OpenSSF compiler hardening guide

11.4 Linters and formatters, one check

Each tool gets one configuration file in the repository. Your editor, the build and CI all read the same file, so they never disagree.

Tool	Checks	Configuration
clang-format	C++ formatting	`.clang-format`
clang-tidy	C++ bug patterns and modern idioms	`.clang-tidy`
qmllint	QML types, names and bindings	`.qmllint.ini`
qmlformat	QML formatting	`.qmlformat.ini`

Wrap the checks in one build target, so “is my code clean?” is a single command, locally and in CI. A second target runs the formatters and fixes the files in place:

```
# clang-format, clang-tidy, qmllint and REUSE
cmake --build --preset dev --target check

# clang-format and qmlformat, fixing the files in place
cmake --build --preset dev --target format
```

Template: `cmake/Checks.cmake`, which also checks licensing with REUSE (Section 13.3)

12. Testing

The layering from Chapter 1. pays off here. Each layer can be tested on its own, most of them without a window, and the tests run in seconds.

12.1 Test each layer without the UI

Layer	Test with	Needs a window
Core (plain C++)	any C++ test framework, such as Catch2	no
View models and stores	Qt Test	no
QML components	Qt Quick Test	yes, offscreen

A view model test calls commands and checks state and signals, exactly as the view would:

```
class tst_SketchbookViewModel : public QObject
{
    Q_OBJECT

private Q_SLOTS:
    void createAddsAnOpenSketch()
    {
        QTemporaryDir dir;
        SketchStorage storage(dir.path());
        CanvasStore canvas;
        SketchbookViewModel viewModel(storage, canvas);
        QSignalSpy opened(&viewModel, &SketchbookViewModel::openNameChanged);

        viewModel.create();

        QCOMPARE(viewModel.model()->rowCount(), 1);
        QCOMPARE(opened.count(), 1);
    }
};

QTEST_GUILESS_MAIN(tst_SketchbookViewModel)
#include "tst_SketchbookViewModel.moc"
```

Line 8
A fresh folder per test: no test sees another test's files.
Line 12
`QSignalSpy` records every emission of a signal.
Template: `tests/sketchbook/` · **Read more:** Qt Test overview

12.2 Test QML components

Qt Quick Test runs test cases written in QML. A test creates the component, simulates input, and checks properties and signals:

```
import QtQuick
import QtTest
import App.Canvas

TestCase {
```

```
    id: testCase
    name: "StrokeItem"
    width: 200
    height: 200
    visible: true
    when: windowShown

    Component {
        id: itemComponent
        StrokeItem { anchors.fill: parent }
    }

    SignalSpy {
        id: finished
        signalName: "strokeFinished"
    }

    function test_dragReportsThePoints(): void {
        const item = createTemporaryObject(itemComponent, testCase);
        finished.target = item;

        mousePress(item, 10, 10);
        mouseMove(item, 90, 80, -1, Qt.LeftButton);
        mouseRelease(item, 90, 80);

        compare(finished.count, 1);
    }
}
```

Tests run without a screen with `QT_QPA_PLATFORM=offscreen`, which the template sets in its test presets. **Template:** `tests/canvas/tst_StrokeItem.qml` · **Read more:** Qt Quick Test

12.3 Tests must be deterministic

A test that fails one run in twenty is worse than no test: people learn to ignore it.

- **Never sleep for a fixed time.** Wait for the condition instead: `QTRY_VERIFY` and `QTRY_COMPARE` in C++, `tryVerify()` and `tryCompare()` in QML.
- **Isolate state.** Temporary directories instead of real paths, fakes at the seams (1) instead of the network or the clock.
- **No order dependence.** Each test sets up what it needs and cleans up after itself.

```
QTest::qWait(500);                  // flaky: sometimes 500 ms is not enough
QTRY_COMPARE(spy.count(), 1);       // waits until true, or fails after a timeout
```

12.4 Check your models

Item models have strict rules: row counts must match the signals, indexes must stay valid, and every change needs the right begin and end calls. `QAbstractItemModelTester` checks all of them while your test runs:

```
SketchListModel model;
QAbstractItemModelTester tester(
    &model, QAbstractItemModelTester::FailureReportingMode::QtTest);
// every change to model is now verified
```

Read more: QAbstractItemModelTester

VI Shipping

13 Shipping 61
13.1 Install, then deploy 61
13.2 One package per platform 61
13.3 Licenses are part of the product 62
13.4 CI builds what you ship 62

14 Signing and distribution 63
14.1 Why sign 63
14.2 macOS: Developer ID and notarization 63
14.3 Windows: Authenticode 63
14.4 Signing in the pipeline 63
14.5 App stores 64

15 Mobile, briefly 65
15.1 Same code, different device 65
15.2 Building for a device 65
15.3 Permissions 65

13. Shipping

An app that only runs on your machine is not done. This chapter turns a build into something a user can download, with everything it needs and every license it owes.

13.1 Install, then deploy

Shipping starts with two steps. `install()` copies your app into a clean folder. Qt's deploy script then copies everything your app needs from Qt next to it: libraries, plugins and QML modules.

```
install(TARGETS app
    BUNDLE DESTINATION .
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)
qt_generate_deploy_qml_app_script(TARGET app OUTPUT_SCRIPT deploy_script)
install(SCRIPT ${deploy_script})
```

```
cmake --install build/release --prefix dist
```

`dist/` now runs on a machine without Qt: a self-contained `.app` bundle on macOS, a folder with the executable and its DLLs on Windows, a `bin/` and `lib/` tree on Linux.

> ⚠ "Could not find or load the Qt platform plugin"
>
> The most famous Qt error means the deploy step did not run, or ran without the plugins. Qt loads plugins at run time, so a build that starts on your machine can still fail on another one.

On Windows, the deploy script knows Qt's DLLs but not those of your other dependencies. Install them too, with `install(FILES $<TARGET_RUNTIME_DLLS:app> ...)`.
Template: `src/app/CMakeLists.txt` · **Read more:** Deploying Qt applications with CMake

13.2 One package per platform

CPack, part of CMake, turns the installed app into the format each platform expects:

Platform	Format	CPack generator
macOS	disk image (`.dmg`)	`DragNDrop`
Windows	archive or installer	`ZIP`, `NSIS`, `WIX`
Linux	archive or package	`TGZ`, `DEB`, `RPM`

```
set(CPACK_PACKAGE_FILE_NAME
    "${PROJECT_NAME}-${PROJECT_VERSION}-${CMAKE_SYSTEM_NAME}")
if(APPLE)
    set(CPACK_GENERATOR DragNDrop)
elseif(WIN32)
    set(CPACK_GENERATOR ZIP)
else()
    set(CPACK_GENERATOR TGZ)
endif()
include(CPack)
```

With a package preset, `cmake --workflow --preset release` configures, builds, tests and packages in one go. Before you hand the package to users, sign it (Chapter 14.).

Template: `cmake/Packaging.cmake` · **Read more:** CPack

13.3 Licenses are part of the product

Every library and asset you ship comes with a license, and most licenses ask for something in return.

Qt is available under the LGPLv3 (and a commercial license). For an app you distribute, the LGPL asks you to:

- let users replace Qt with their own build; linking Qt dynamically is the simple way;
- ship Qt's license texts and say that your app uses Qt;
- provide Qt's source code, or a written offer for it;
- avoid the Qt modules that are available only under the GPL.

Every other dependency and asset needs its license shipped as well: libraries, but also fonts and icons. Put one text file per component into a `licenses/` folder of the package.

Your own repository should say clearly what is yours and under which terms. The REUSE convention does that in one file:

```
version = 1

[[annotations]]
path = "**"
SPDX-FileCopyrightText = "2026 Your Name"
SPDX-License-Identifier = "MIT-0"

[[annotations]]
path = "src/style/fonts/**"
SPDX-FileCopyrightText = "2016 The Inter Project Authors"
SPDX-License-Identifier = "OFL-1.1"
```

`reuse lint` then checks that every file in the repository is covered.

This is a summary of obligations, not legal advice.

Template: `REUSE.toml`, `app_add_notice()` in `cmake/Packaging.cmake`, and the About dialog · **Read more:** Qt licensing

13.4 CI builds what you ship

Every change should build, test and package on every platform you support, on a machine that is not yours. Use the same preset command that developers run:

```
jobs:
  build:
    strategy:
      matrix:
        os: [ubuntu-24.04, macos-15, windows-2022]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v5
        with:
          submodules: true
      - uses: jurplel/install-qt-action@v4
        with:
          version: "6.12.0"
          cache: true
      - run: cmake --workflow --preset release
```

A complete workflow also sets up the compiler environment on Windows. Cache the dependency builds between runs (vcpkg's binary cache fits here), or every run rebuilds them from scratch. Upload the packages as build artifacts, so every change produces something you can download and try.

Template: `.github/workflows/ci.yml`

14. Signing and distribution

A package is ready to ship when the user's operating system trusts it. On macOS and Windows, that trust comes from a signature.

14.1 Why sign

macOS and Windows check where downloaded software comes from. An unsigned app gets a stern warning on Windows (SmartScreen) and is blocked outright on macOS (Gatekeeper) until the user overrides it.

A signature proves two things: **who** published the app, and that **nobody changed it** since. Unsigned builds are fine for development and a course project. Anything you hand to users should be signed.

14.2 macOS: Developer ID and notarization

Apps distributed outside the Mac App Store need three things:

1. **A Developer ID certificate**, from the Apple Developer Program.
2. **A signature on every binary**, made with the *hardened runtime* and a secure timestamp.
3. **Notarization**: you upload the app to Apple, Apple scans it and issues a ticket, and you *staple* the ticket to the app so Gatekeeper accepts it even offline.

Qt's deploy tool signs the bundle for you when you pass it an identity. Use the certificate's SHA-1 hash, which `security find-identity -v -p codesigning` lists: the deploy script would split a name with spaces into several arguments.

```
qt_generate_deploy_qml_app_script(TARGET app OUTPUT_SCRIPT deploy_script
    DEPLOY_TOOL_OPTIONS "-sign-for-notarization=${APP_SIGNING_IDENTITY}"
)
```

Then sign the disk image, notarize it and staple the ticket:

```
codesign --sign "Developer ID Application: Your Name (TEAMID)" \
    --timestamp Sketchbook.dmg
xcrun notarytool submit Sketchbook.dmg --keychain-profile notary --wait
xcrun stapler staple Sketchbook.dmg
```

Read more: Qt for macOS: deployment, Apple: notarizing macOS software

14.3 Windows: Authenticode

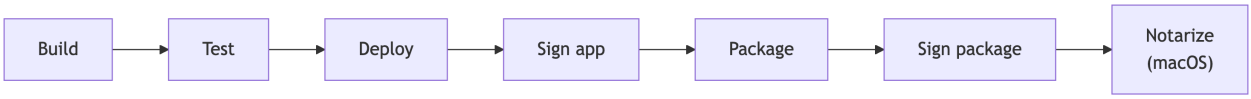
Windows signatures are called Authenticode. You sign the executable and the installer with `signtool` from the Windows SDK:

```
signtool sign /fd SHA256 /td SHA256 /a `
    /tr http://timestamp.digicert.com Sketchbook.exe
signtool verify /pa Sketchbook.exe
```

The certificate comes from a certificate authority or from a cloud signing service, such as Microsoft's. Always add a timestamp (`/tr`): it keeps the signature valid after the certificate expires. SmartScreen builds trust in a publisher over time, as more users install its signed releases.
Read more: Microsoft: code signing options, SignTool

14.4 Signing in the pipeline

The order matters:



- **Sign after deploying.** The deploy tool still changes binaries, and any change breaks a signature.
- **Sign inside out.** Libraries and plugins first, then the app, then the disk image or installer that contains it.
- **Treat certificates as secrets.** They live in your CI's secret store, never in the repository, and only release builds use them.

14.5 App stores

App stores take over trust and updates, in exchange for their rules: a sandbox, a review, a store package format.

Store	Platform	Notes
Mac App Store	macOS	App Sandbox required
Microsoft Store	Windows	MSIX packages; `windeployqt --appx` prepares one (Qt 6.11+)
Google Play, App Store	Android, iOS	the normal way to ship mobile apps (Chapter 15.)

Read more: Microsoft Store deployment with windeployqt

15. Mobile, briefly

The same Qt Quick application can run on Android and iOS. This chapter does not teach mobile development; it shows what changes, so you can plan for it.

15.1 Same code, different device

What stays: your C++ layers, your QML, your build. What changes is the device around them:

- **Input.** Fingers instead of a mouse: no hover, larger touch targets, gestures.
- **Screens.** Small, dense, and rotating between portrait and landscape.
- **Lifecycle.** The system suspends your app when the user switches away, and may end it without warning. Save early, not on quit.
- **Files.** Each app lives in a sandbox; use `QStandardPaths` (Section 10.2), never fixed paths.

This is where the split from Section 1.1 pays off once more: logic stays in C++, and mostly the views adapt.
Read more: Mobile development with Qt

15.2 Building for a device

Mobile builds are cross-compiled with the platform's own tools: the Android SDK and NDK, or Xcode for iOS. Qt provides a build for each platform, and the Qt Online Installer installs them next to your desktop Qt. vcpkg builds your other dependencies for mobile triplets from the same `vcpkg.json` (Section 4.3).
Read more: Getting started with Qt for Android, Qt for iOS

15.3 Permissions

The camera, location, microphone or Bluetooth need the user's consent, asked at run time. Qt offers one permission API for all platforms:

```
import QtCore

CameraPermission {
    id: camera
}

Button {
    text: qsTr("Scan a sketch")
    visible: camera.status !== Qt.PermissionStatus.Denied
    onClicked: camera.status === Qt.PermissionStatus.Granted
               ? scanner.start() : camera.request()
}
```

Each permission must also be declared: in the app's `AndroidManifest.xml` on Android, and with a usage description that tells the user why in `Info.plist` on iOS. Ask for as little as you need, at the moment you need it, so the user understands why.
Read more: Application permissions

VII Beyond

16 Learning Qt, and working with AI . . 69
16.1 Concepts you own, details you look up 69
16.2 Read in the right order 69
16.3 Recognize stale Qt 69
16.4 Give the AI current Qt knowledge 69
16.5 Let the tools judge the result 70
16.6 Own what you ship 70

16. Learning Qt, and working with AI

Qt is large, it changes every release, and most of what the internet knows about it is out of date. This last chapter is about learning it efficiently and about using AI assistants without being misled by them.

16.1 Concepts you own, details you look up

Some things you must understand yourself: what a binding does, why the event loop must not block, who owns an object. Without them you cannot tell a good answer from a plausible one, whether it comes from a colleague, a forum or an AI.

Other things nobody memorizes: the argument order of a function, the name of an option. Look them up, every time. This guide teaches the first kind and links to the second.

16.2 Read in the right order

When you meet a new Qt module, spend fifteen minutes in this order:

1. The module's overview page: what problem it solves, which classes matter.
2. One example, end to end.
3. The reference pages of the two or three classes that example used.
4. Now write your own code.

Qt's own examples and tests show current, working usage. When the documentation leaves a question open, the test suite of that class often answers it.

16.3 Recognize stale Qt

Most Qt code online, and in the data AI models learned from, is Qt 5. It often still compiles, but it is not how Qt 6 works. Learn its tells:

Qt 5	Qt 6
`qmake` and `.pro` files	CMake with `qt_add_qml_module()`
`qmlRegisterType<T>("Uri", 1, 0, "T")`	`QML_ELEMENT` in the class
`setContextProperty("name", &object)`	required properties (Section 7.2)
`connect(a, SIGNAL(x()), b, SLOT(y()))`	`connect(a, &A::x, b, &B::y)`
`import QtQuick 2.15`	`import QtQuick`
`onClicked: ...` inside `Connections`	`function onClicked() { ... }` inside `Connections`

When an answer contains one of these, check it against the Qt 6 documentation before you use it.

16.4 Give the AI current Qt knowledge

A coding agent repeats what it learned unless it gets better context. The Qt Company publishes **agent skills** for the common AI coding tools. They correct the known mistakes and add Qt-specific workflows:

Skill	What it does
`qt-qml`	QML best practices; corrects typical mistakes with bindings, scoping, modules and types

Skill	What it does
`qt-cmake-project`	Qt 6 CMake projects; corrects qmake habits and old `qt5_*` macros
`qt-qml-review`, `qt-cpp-review`	code review with linters plus deeper analysis
`qt-qml-test`	writes Qt Quick Test cases for a component
`qt-ui-design`	layout, navigation and UX review

The same repository provides a **Qt documentation server** (an MCP server) that lets the agent look up the current API instead of guessing it.
Read more: Qt agent skills

16.5 Let the tools judge the result

An agent is only as good as the feedback it gets. Compiler warnings, qmllint, tests and sanitizers give it ground truth instead of its own confidence. A project where one command runs every check is a project an agent can work in:

```
cmake --workflow --preset dev                # build and run every test
cmake --build --preset dev --target check    # every linter
```

The quality gates of Chapter 11. and the tests of Chapter 12. are the same gates you want for human contributors. For UI work, a live preview (Qt offers one as an MCP tool) lets the agent see a change instead of describing it.
Read more: QML Live Preview MCP tool

16.6 Own what you ship

Review generated code the way you would review a colleague's: read every line, run it, test it. Keep only what you can explain.

> Rule of thumb
>
> If you could not rewrite it from scratch with an hour and the documentation, you do not understand it yet. Go back to the concept.

Appendices

A. Working with the template

Short recipes for common changes to the template. Each assumes you have read the chapter it refers to.

A.1 Rename the template

1. In the top-level `CMakeLists.txt`, change the name in `project()`, `APP_DISPLAY_NAME` and `APP_BUNDLE_ID`.
2. Replace the QML prefix `App` and the C++ namespace `app`, for example with `Acme`:

```
rg -l 'App\.|"App"|URI App\b|AppStyle|app::|namespace app\b' src tests \
    | xargs perl -pi -e 's/\bApp\./Acme./g; s/"App"/"Acme"/g; s/URI App\b/URI Acme/;
s/\bAppStyle\b/AcmeStyle/g; s/\bapp::/acme::/g; s/namespace app\b/namespace acme/g'
```

3. Put your copyright and license into `LICENSE` and the first entry of `REUSE.toml`, then build and run `check`.

A.2 Remove the demo

1. Delete `src/canvas/`, `src/sketchbook/`, `tests/canvas/`, `tests/sketchbook/` and `libs/shapes/`, and their `add_subdirectory()` lines.
2. In `src/app/CMakeLists.txt`, remove the `app_canvas*` and `app_sketchbook*` entries.
3. In `main.cpp`, remove the storage, the store and the view model; in `Main.qml`, remove the two required properties, their imports and the pages.
4. Remove `glaze` from `vcpkg.json` and the top-level `CMakeLists.txt` if nothing else uses it.

Keep `src/style`: it is the base for your own views.

A.3 Add a separable library

See Section 2.3.

1. Create `libs/<name>/` with `include/<name>/`, `src/` and `tests/`, and a `CMakeLists.txt` modeled on `libs/shapes/CMakeLists.txt`. Call `app_target_defaults(<target>)` so the library gets warnings, hardening and checks.
2. Add `add_subdirectory(libs/<name>)` to the top-level `CMakeLists.txt`, before `src`.
3. Link it from the feature that uses it: `target_link_libraries(app_<feature> PUBLIC <name>::<name>)`.

A.4 Add a feature

See Section 2.4 and Section 7.2.

1. Create `src/<feature>/` with its C++ types, marked `QML_ELEMENT` and `QML_UNCREATABLE`, its pages, and a `qt_add_qml_module(app_<feature> URI App.<Feature> STATIC ...)`. Link `app_style` privately if its QML imports `AppStyle`.
2. Add `add_subdirectory(<feature>)` to `src/CMakeLists.txt`, before `app`.
3. Link `app_<feature>` and `app_<feature>plugin` in `src/app/CMakeLists.txt`.
4. In `main.cpp`, construct the feature's object before the engine and add it to `setInitialProperties()`; in `Main.qml`, declare it as a required property and pass it to the page.
5. Add `tests/<feature>/` with Qt Test or Qt Quick Test, and `add_subdirectory(<feature>)` to `tests/CMakeLists.txt`.

A.5 Add a dependency from vcpkg

See Section 4.3.

1. Add the port to `dependencies` in `vcpkg.json`.
2. Add `find_package(<Package> CONFIG REQUIRED)` to the top-level `CMakeLists.txt`, and link the target where you need it. The port's usage text, printed during configure, names the targets.
3. Reconfigure. Its license ships in the package's `licenses/` folder automatically.

A.6 Ship a license notice for an asset

See Section 13.3. Put the asset's upstream license file next to it, register it in that folder's `CMakeLists.txt`, and add the asset to `REUSE.toml`:

```
app_add_notice(lucide-icons icons/LICENSE)
```

The package then contains `licenses/lucide-icons.txt`.

A.7 Change the app icon

`src/app/app.svg` is the only icon you design; the About dialog shows it as is. macOS and Windows need the icon in their own formats, `app.icns` and `app.ico`, so regenerate those two after changing the SVG. On macOS, with ImageMagick for the `.ico`:

```
cd src/app
mkdir app.iconset
for s in 16 32 128 256 512; do
    sips -s format png -Z $s app.svg --out app.iconset/icon_${s}x${s}.png
    sips -s format png -Z $((s * 2)) app.svg --out app.iconset/icon_${s}x${s}@2x.png
done
iconutil -c icns app.iconset && rm -r app.iconset
magick -background none app.svg -define icon:auto-resize=256,64,48,32,16 app.ico
```

A.8 Add a language

See Section 10.1.

1. Add the language code to `I18N_TRANSLATED_LANGUAGES` in the top-level `CMakeLists.txt`.
2. Generate the new `.ts` file and translate it, for example with Qt Linguist:

   ```
   cmake --build --preset dev --target update_translations
   linguist i18n/app_fr.ts
   ```

3. Rebuild. The `.qm` file is embedded automatically, together with Qt's own translations for that language, and `AppController.language = "fr"` switches to it at run time.

A.9 Debug with more logging

See Section 10.4.

```
QT_LOGGING_RULES="app.*.debug=true" \
    build/dev/src/app/QtAppTemplate.app/Contents/MacOS/QtAppTemplate
```

On Windows and Linux, the executable is `build/dev/src/app/QtAppTemplate`. The session logs are in the folder that *About, Open log folder* opens.